

# Comprehensive Study Notes: Paged KV Cache, RoPE, and vLLM vs. Unsloth

---

These notes explain modern LLM inference architectures, mathematical position representations, and system-level orchestration in accessible, practical language.

## 1. Paged KV Cache (PagedAttention)

### What is the KV Cache?

When an autoregressive transformer generates text, it produces tokens one by one. In attention computation, each new token must attend to all previous tokens. To avoid recalculating the Key (K) and Value (V) representations of older tokens at every generation step, we store them in GPU VRAM. This saved memory is the **KV Cache**.

### The Problem: Memory Fragmentation in Traditional Serving

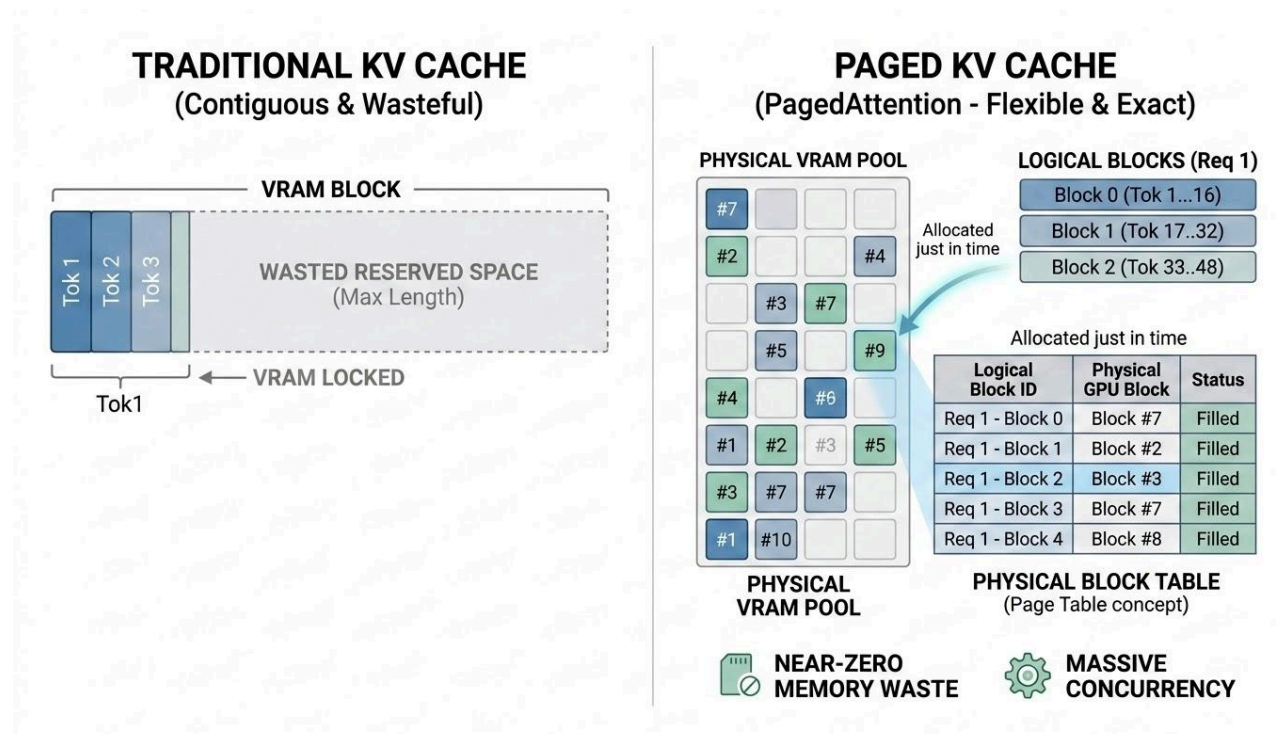
In standard serving frameworks (like naive Hugging Face Transformers), memory is allocated ahead of time as contiguous chunks based on the worst-case maximum sequence length (e.g., reserving space for 4096 tokens even if the user only generates 40). This causes three major issues:

- **Internal Fragmentation:** Unused pre-allocated memory inside reserved buffers.
- **External Fragmentation:** Gaps between allocations that cannot fit incoming requests.
- **Over-reservation:** Engines end up wasting 60% to 80% of GPU memory, capping request concurrency at very low numbers.

### The Solution: PagedAttention

Inspired by **Virtual Memory and Paging in Operating Systems**, PagedAttention splits each request's KV cache into small, fixed-size chunks called **blocks** (typically 16 or 32 tokens per block). These blocks do not need to be stored in contiguous physical memory addresses on the GPU.

### Architectural Diagram: Traditional vs. PagedAttention



## Key Benefits

- **Near-Zero Memory Waste:** Drops memory waste down to less than 4% (only the tiny tail end of the active block).
- **Massive Concurrency:** Packing 2x to 4x more active requests into the same GPU VRAM.
- **Copy-on-Write Sharing:** Parallel sampling (e.g. generating 4 responses to 1 prompt) or multi-turn prompt caching can share the exact same prompt blocks without duplicating VRAM.

## 2. RoPE (Rotary Position Embedding)

### Why Do Transformers Need Position Embeddings?

Standard self-attention is permutation-invariant: it treats tokens as an unordered set of words. Without position signals, "dog bites man" and "man bites dog" look identical to the attention mechanism.

### Traditional vs. RoPE

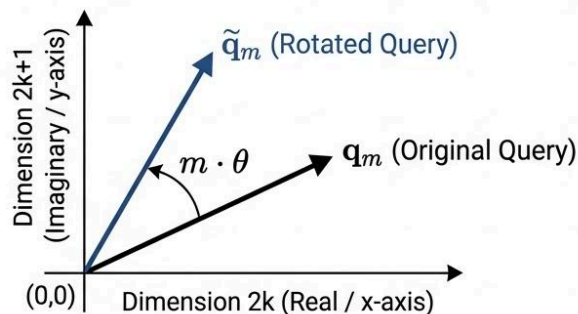
- **Absolute Position Embeddings (e.g., BERT, GPT-2):** Adds a position vector directly to token embeddings:  $\text{Input} = \text{Token\_Embed} + \text{Pos\_Embed}$ . This approach struggles to generalize to sequence lengths beyond what was seen during training.
- **Relative Position Embeddings:** Adds relative bias terms into attention matrices, but often introduces substantial computational overhead.

- **RoPE (Rotary Position Embedding):** Multiplies the query and key vectors by an orthogonal 2D rotation matrix proportional to the token's position in the sequence.

### Geometric Intuition: Vector Rotation

RoPE pairs up adjacent feature dimensions of the Query (q) and Key (k) vectors (dim 0 with dim 1, dim 2 with dim 3, etc.) and rotates each 2D slice on a 2D plane by an angle equal to  $m \cdot \theta$ , where  $m$  is the token's index position.

### Rotary Position Embedding (RoPE) 2D Vector rotation



### GEOMETRIC INTUITION & KEY EQUATIONS

$$\begin{aligned}
 \text{Score}(m, n) &= \langle \tilde{\mathbf{q}}_m, \tilde{\mathbf{k}}_n \rangle = \mathbf{q}^T \cdot R(n - m) \cdot \mathbf{k} \\
 &= \text{Rotated}(\mathbf{q}, m) \cdot \text{Rotated}(\mathbf{k}, n) \\
 &= \mathbf{q}^T \cdot R(m)^T \cdot R(n) \cdot \mathbf{k}
 \end{aligned}$$

The absolute positions  $(m, n)$  cancel out!  
attention depends solely on relative distance  $(m - n)$ .

### Why RoPE Is Ubiquitous (LLaMA, Mistral, Qwen, Gemma)

- **Decays Naturally with Distance:** As the distance  $|m - n|$  grows, the rotation angles de-correlate, naturally focusing the attention score on local context.
- **Seamless Extrapolation (Long Context):** By adjusting rotation frequencies (techniques such as RoPE scaling, YaRN, or NTK-aware interpolation), models trained on 4k or 8k context windows can be scaled up to 32k, 64k, or 128k context windows with negligible fine-tuning.

## 3. vLLM and Unsloth: Roles, Mechanisms, and Differences

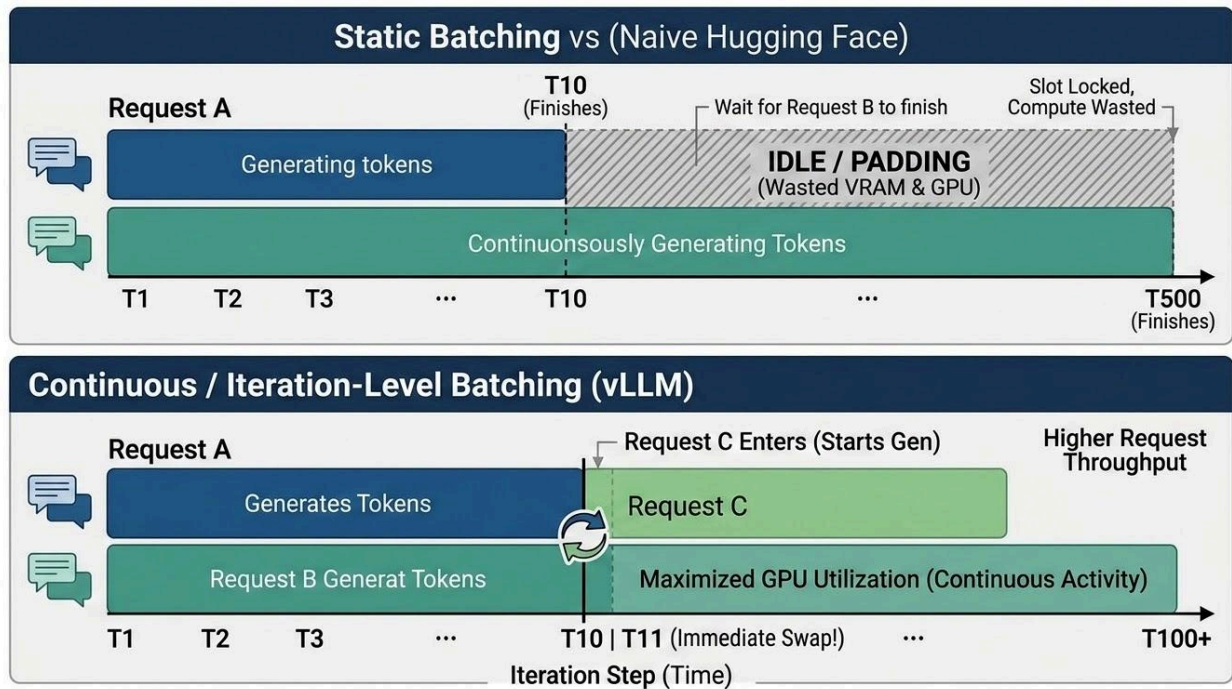
## Core Distinction

**Unsloth** is designed for *training and fine-tuning*, while **vLLM** is designed for *production inference serving*. They tackle different bottlenecks in the LLM lifecycle.

| Dimension         | Unsloth                                                                                                                | vLLM                                                                            |
|-------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Primary Purpose   | Super-fast, memory-efficient fine-tuning (LoRA, QLoRA, Full).                                                          | High-throughput, multi-user production inference serving.                       |
| Core Innovation   | Custom Triton and CUDA kernels hand-written to accelerate forward/backward passes and eliminate autograd memory bloat. | PagedAttention engine, Continuous Iteration-Level Batching, Tensor Parallelism. |
| Concurrency Model | Single-process, batch gradient optimization over static datasets.                                                      | Multi-client asynchronous HTTP/gRPC server handling high concurrency.           |
| Output Artifact   | Trained adapter weights, merged 16-bit safetensors, or GGUF files.                                                     | Streaming text tokens via OpenAI-compatible endpoints.                          |
| Memory Focus      | Minimizing activation cache and gradient memory footprint during training.                                             | Zero fragmentation of KV cache across hundreds of concurrent live users.        |

## Why Continuous Batching in vLLM Matters

Traditional static batching forces incoming user queries to wait until all requests in the current batch have finished generating all tokens. If User A asks for 10 tokens and User B asks for 500 tokens, User A's GPU slot sits completely idle for 490 iterations.



## 4. The End-to-End Practical Workflow

In industry and research setups, Unsloth and vLLM are complimentary tools in the pipeline:

### 1. Step 1: Train with Unsloth

Load your base model (e.g., LLaMA-3, Mistral, Qwen) using Unsloth. Fine-tune your custom dataset 2x to 5x faster using 70% less VRAM.

### 2. Step 2: Export Weights

Merge the trained LoRA adapters into standard 16-bit Hugging Face safetensors format.

### 3. Step 3: Serve with vLLM

Launch vLLM using the exported weights directory. vLLM provides continuous batching, PagedAttention, and an OpenAI-compatible API endpoint (/v1/chat/completions) capable of serving live web applications.

**Key Takeaway:** Use **Unsloth** to build and adapt the brain quickly; use **vLLM** to serve that brain efficiently to hundreds of users concurrently.